

Repository Management Dashboard

Immediate Migration Checklist

1. Baseline all three source repos and capture current outputs ([doc-review-manifest.json](#), reconciliation JSON/CSV, dashboard screenshots/API responses).
2. Extract shared PowerShell modules for scanning, git metadata, and report generation into a new [backend/](#) layout.
3. Stand up a single local API surface for [status](#), [settings](#), [update](#), [sync](#), [export](#), [reconcile](#), and [doc-review](#) orchestration behind feature flags.
4. Switch the UI to the unified API contract while preserving the existing local-only workflow.
5. Run parity tests and manual operator flows before cutover, then keep rollback scripts ready for one-command reversion.

Consolidates three Windows-first repository management tools into one maintainable dashboard:

- [G:\Development\20_Staging\GitHubRepoManagerDashboard](#)
- [G:\Development\Doc_Review_Inventory](#)
- [G:\Development\Repo_reconciliation-dashboard](#)

The target outcome is a single operator-facing dashboard with a modular PowerShell/.NET backend, a lightweight frontend, and strong observability.

What This Consolidation Delivers

- Unified repository inventory, local/git health, and GitHub reconciliation views.
- Integrated documentation review planning pipeline (inventory -> queue -> per-repo batches).
- Standardized report outputs (JSON/CSV/HTML/Markdown) and reusable machine-readable contracts.
- Shared error handling, logging, metrics, and health checks.
- Sequenced migration path that preserves partial usability throughout transition.

See [Consolidated Features](#) for the domain-by-domain feature set.

See [Canonical Contracts](#) for data and error envelope definitions.

Architecture Snapshot

The target system keeps Windows-first execution with PowerShell/.NET as the backend core and a simple web UI:

- Backend orchestration modules (inventory, reconciliation, doc-review, reports).
- Unified HTTP API for UI and automation.
- Optional scheduled execution (Task Scheduler) for periodic scans and report generation.
- Structured logs + metrics + health endpoints.

See [Architecture and Design](#) for full component and flow details.

Prerequisites and Environment Assumptions

- Windows workstation/home-lab environment.
- PowerShell runtime installed.
- .NET runtime/SDK available for API host and backend services.
- Git CLI available in **PATH**.
- Optional: GitHub CLI (**gh**) for remote reconciliation and owner inventory.

TODO:

- Verify exact PowerShell runtime baseline for production (**5.1** vs **7+**) and command compatibility.
- Verify .NET target and build/runtime profile for final backend host.
- Verify minimum dependency versions after lockfile/audit pass.

Repository Layout

```
GitHubRepoManagement/  
├── README.md  
├── CONTRIBUTING.md  
├── CODE_OF_CONDUCT.md  
├── SECURITY.md  
├── CHANGELOG.md  
├── .github/  
│   ├── CODEOWNERS  
│   ├── PULL_REQUEST_TEMPLATE.md  
│   ├── ISSUE_TEMPLATE/  
│   └── workflows/  
├── standards/  
└── docs/  
    ├── index.md  
    ├── architecture/  
    ├── planning/  
    ├── reference/  
    ├── operations/  
    └── archive/
```

Installation and Setup

1. Clone and prepare workspace

```
# Clone this consolidation workspace  
# Replace URL with your remote  
git clone <repo-url> G:\Development\GitHubRepoManagement  
  
Set-Location G:\Development\GitHubRepoManagement
```

2. Validate source repos are present

```
# Confirm all source repositories are available before migration
$paths = @(
    'G:\Development\20_Staging\GitHubRepoManagerDashboard',
    'G:\Development\Doc_Review_Inventory',
    'G:\Development\Repo_reconciliation-dashboard'
)

$paths | ForEach-Object {
    if (Test-Path $_) { Write-Host "[OK] $_" -ForegroundColor Green }
    else { Write-Host "[MISSING] $_" -ForegroundColor Red }
}
```

3. Backend/frontend build-run guidance

The current source repos use mixed stacks (PowerShell scripts, Node/React UI, planned .NET backend). For the consolidated target:

- Keep backend modules runnable independently during migration.
- Introduce a single API host layer when contract stabilization begins.
- Keep frontend lightweight (existing React UI can be retained or simplified to plain HTML/JS if preferred).

TODO:

- Finalize canonical build commands once target backend project scaffolding is committed.
- Finalize release packaging layout for single-machine deployment.

4. Configuration and secrets

- Keep operator settings in a local config file (non-secret) under a known path.
- Keep tokens (GitHub PAT, API keys) out of source control and out of browser local storage.
- Pass secrets at runtime via environment variables or secure host-level secret store.

Example:

```
# Example runtime token injection for current session only
$env:GITHUB_TOKEN = '<token>' # Do not persist in script files
# Launch backend host after env var is set
```

Usage Workflows

Run local repository inventory and reconciliation

```
Set-Location 'G:\Development\Repo_reconciliation-dashboard'

# Local + GitHub owner reconciliation
```

```
.\src\repo_reconciliation_dashboard.ps1 `
  -LocalRoots @('G:\Development') `
  -GitHubOwner 'xfaith4' `
  -LogPath '.\logs\reconciliation_run.log' # Persist detailed execution logs
```

Run documentation review pipeline

```
Set-Location 'G:\Development\Doc_Review_Inventory'

# Stage 1: Build markdown inventory manifest
.\scripts\Invoke-DocReviewInventory.ps1 `
  -RootPath 'G:\Development\20_Staging' `
  -OutDir '.\output\inventory' `
  -MaxDepth 3 # Limit scan depth for predictable runtime

# Stage 2a: Build cross-repo review queue
.\scripts\Build-DocReviewQueue.ps1 `
  -ManifestPath '.\output\inventory\doc-review-manifest.json' `
  -OutDir '.\output\queue' `
  -MaxFilesPerBatch 5
```

Run dashboard local status scan

```
Set-Location 'G:\Development\20_Staging\GitHubRepoManagerDashboard'

# Starts frontend + backend live mode for local repository status operations
.\start-live.bat # Uses backend API for pull/fetch/status operations
```

Troubleshooting and Diagnostics

- Enable verbose PowerShell output during migration scripts:

```
$VerbosePreference = 'Continue' # Increase script verbosity in current session
```

- Use preflight checks before large reconciliation runs:

```
Set-Location 'G:\Development\Repo_reconciliation-dashboard'
.\tests\preflight-check.ps1 # Validates gh auth, PowerShell, and path access
```

- Validate doc pipeline manifest quality before queue generation:

```
Set-Location 'G:\Development\Doc_Review_Inventory'  
Get-Content '.\output\inventory\doc-review-report.md' -TotalCount 80
```

Observability Summary

Target baseline:

- Structured logs: console + rotating file sink (and optional ETW sink on Windows).
- Metrics: operation counters, scan duration histograms, queue depth gauges, error counters.
- Health endpoints: liveness/readiness/dependency checks for API host and job runners.
- API metrics route: `GET /metrics`.
- Artifact index route: `GET /api/report/artifacts`.

Defined in detail in [Architecture Observability](#).

Maintenance and Next Documents

- [Docs Index](#)
- [Architecture and Design](#)
- [Migration Plan](#)
- [Unified Roadmap](#)
- [Consolidated Features](#)
- [Canonical Contracts](#)
- [Architecture Decisions](#)
- [Outstanding TODOs and Source Mapping](#)

Operational Scripts

```
# API host smoke including dependencies/metrics/artifacts routes  
.\scripts\Invoke-APIHostSmokeTest.ps1  
  
# Register daily scheduled tasks for status + reconcile  
.\scripts\Register-ScheduledTasks.Template.ps1 -WorkspaceRoot  
'G:\Development\GitHubRepoManagement' -TaskPrefix 'RepoMgmt'  
  
# Cleanup artifacts/logs by retention age  
.\scripts\Invoke-RetentionCleanup.ps1 -WorkspaceRoot  
'G:\Development\GitHubRepoManagement' -RetentionDays 30
```